NgRx DevTools — Complete Guide

Chrome · Angular 17 Standalone · From Zero to Time-Travel Debugging

What Is NgRx DevTools and Why Do You Need It?

When your Angular app is running, the NgRx Store is just an object in memory — invisible to you. Without DevTools you have no way to answer these questions:

- What is the **current value** of my store right now?
- Which **actions fired** in the last 10 seconds?
- What did the state look like **before** a specific action?
- **What exactly changed** as a result of that action?
- Which component **dispatched** the action that caused a bug?

NgRx DevTools answers all of these. It connects your running Angular app to the **Redux DevTools browser extension** and gives you a live panel inside Chrome DevTools.

> **Two pieces are needed — both are required:**
>
> **1. The browser extension** — installed once in Chrome. It provides the panel UI inside Chrome DevTools.

> **2. The npm package** (`@ngrx/store-devtools`) — installed in your project. It is the bridge that sends your app's actions and state to the browser extension. Without this package, the extension has nothing to display.

---

1 Install the Browser Extension

Step A — Go to the Chrome Web Store

Open Chrome and navigate to this URL:

```
https://chrome.google.com/webstore/detail/redux-devtools/lmhkpmbekcpmknklioeibfkpmmfibljd
```

Or search **"Redux DevTools"** in the Chrome Web Store and look for the extension by *remotedev.io*.

Step B — Click "Add to Chrome"

A permission popup will appear:

```
"Redux DevTools" wants to:
  - Read and change all your data on websites you visit
```

Click **"Add extension"**. This permission is what allows the extension to communicate with the NgRx store running on your `localhost` page. Without it the extension cannot see your app's state.

Step C — Confirm the Icon Appears

After installing, a small icon appears in your Chrome toolbar — it looks like a red/orange atom symbol.

> **The icon will be grey on most websites.** It only becomes colourful when it detects a Redux-compatible store on the current page. It will light up on your Angular app after we complete Part 3. If it stays grey, the npm package is not yet wired in.

---

2 Install the npm Package

In your project folder (`ngrx-user-profile`), run:

```
npm install @ngrx/store-devtools@17
```

Verify it was added:

```
# Check package.json — you should see:
# "@ngrx/store-devtools": "^17.x.x"

# Or run:
npm list @ngrx/store-devtools
```

> **Why is a separate package needed?**
> The browser extension is generic — it works with any Redux-based library (plain Redux, NgRx, Vuex, etc.). The `@ngrx/store-devtools` package is the Angular-specific adapter that serialises your NgRx store and streams it to the extension in real time.

---

3 Wire It Into Your Angular App

This is a **one-file change**. Open `src/app/app.config.ts` and add `provideStoreDevtools()` to the providers array.

Before (what you already have):

`src/app/app.config.ts — BEFORE`

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideStore } from '@ngrx/store';

import { routes } from './app.routes';
import { userProfileReducer } from './state/user-profile.reducer';
```

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideStore({
      userProfile: userProfileReducer,
    }),
  ],
};
```

After (with DevTools added):

src/app/app.config.ts — AFTER

```
import { ApplicationConfig, isDevMode } from '@angular/core';  // ← add isDevMode
import { provideRouter } from '@angular/router';
import { provideStore } from '@ngrx/store';
import { provideStoreDevtools } from '@ngrx/store-devtools';    // ← new import

import { routes } from './app.routes';
import { userProfileReducer } from './state/user-profile.reducer';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideStore({
      userProfile: userProfileReducer,
    }),

    // ── Add this block ──────────────────────────────
    provideStoreDevtools({

      // How many past actions to keep in the DevTools history log.
      // Once exceeded, the oldest entries are dropped.
      maxAge: 25,

      // isDevMode() returns true during `ng serve` (development)
      // and false during `ng build` (production).
      //
      // logOnly: true  → DevTools is read-only, cannot modify state
      // logOnly: false → DevTools is fully active
      //
      // So: fully active in dev, safely read-only in production.
      logOnly: !isDevMode(),
```

```
      // Keep this true — enables the connection to the browser extension.
      autoPause: true,

      // A display name shown inside the DevTools panel.
      // Useful if you have multiple Angular apps open at once.
      name: 'NgRx User Profile App',
    }),
    // ——————————————————————————————————————————————
  ],
};
```

Save the file, then run the app:

```
ng serve
```

> Open `http://localhost:4200`. The Redux DevTools icon in your Chrome toolbar should now be **colourful (red/orange)**. This confirms the extension and your app are communicating successfully.

---

4 Open the DevTools Panel

1. Open Chrome and go to `http://localhost:4200`
2. Press `F12` to open Chrome Developer Tools
3. Look at the tab bar at the top of the DevTools panel:

   ```
   Elements   Console   Sources   Network   Performance   ...   >>
   ```

4. Click the `>>` arrow at the right end — it reveals hidden tabs
5. Find "**Redux**" in the dropdown and click it

> **Tip — pin the Redux tab:** Right-click the Redux tab and choose "Move to main panel" so it always appears without clicking `>>`.

Still not seeing the Redux tab? Check these in order:

1. Confirm you saved `app.config.ts` after adding `provideStoreDevtools()`
2. Confirm the dev server restarted (`Ctrl` + `C` then `ng serve` again)
3. Confirm the toolbar icon is colourful, not grey
4. Try closing and reopening the DevTools panel entirely

---

5 The DevTools Panel Explained

The panel is split into two halves. Here is a visual mockup of what you will see:

The Left Side — Action Log

Every action that has been dispatched appears here in chronological order, from oldest at the top to newest at the bottom. Click any entry to inspect it in the right panel.

The very first entry is always `@@INIT`. You did not dispatch it — NgRx dispatches it automatically when your app first loads to initialise the store with the `initialState` you defined in Step 1. Every entry after it is an action your code dispatched.

The Right Side — Inspector Panel (4 Tabs)

Tab	What it shows	When to use it
Action	The full action object — `type` + all payload fields	Confirm the component sent the right data

Tab	What it shows	When to use it
State	The complete store state *after* this action ran	Verify the reducer produced the correct new state
Diff	Only the fields that changed between previous and current state	Fastest way to spot exactly what a single action changed
Test	An auto-generated unit test for this action sequence	Advanced — useful once you start writing reducer tests

---

6 Live Walkthrough — What You Will See

Open your app, open the Redux tab, and follow along. Here is exactly what each step produces in the DevTools panel.

On First Load

The action log shows one entry:

```
@@INIT
```

Click `@@INIT`, then click the **State** tab on the right:

```
{
  "userProfile": {
    "name":  "",
    "email": "",
    "age":   null
  }
}
```

This is your `initialUserProfileState` — exactly what you wrote in `user-profile.state.ts`. The store exists and is waiting for actions.

After Filling the Form and Clicking "Save to Store"

A new entry appears in the action log:

```
@@INIT
[Profile Form] Save Profile   ← new
```

Click **[Profile Form] Save Profile** and inspect each tab:

Action tab — the raw object that was dispatched:

```
{
  "type":  "[Profile Form] Save Profile",
  "name":  "Reza",
  "email": "reza@example.com",
  "age":   25
}
```

State tab — the full store AFTER the reducer ran:

```
{
  "userProfile": {
    "name":  "Reza",
    "email": "reza@example.com",
    "age":   25
  }
}
```

Diff tab — only what changed:

```
userProfile
  name:   "" → "Reza"
  email: "" → "reza@example.com"
  age:   null → 25
```

In a large app with dozens of state slices, the Diff tab is the fastest way to see the impact of a single action — it filters out everything that did not change.

After Clicking "Clear Profile"

```
@@INIT
[Profile Form] Save Profile
[Profile Form] Clear Profile   ← new
```

Action tab — no payload on a clear action:

```
{
  "type": "[Profile Form] Clear Profile"
}
```

Diff tab:

```
userProfile
  name:   "Reza" → ""
  email: "reza@example.com" → ""
  age:   25 → null
```

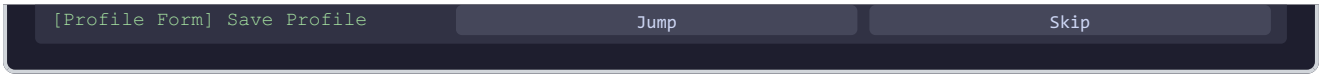
---

7 Time-Travel Debugging

> **This is the feature that makes DevTools genuinely powerful.** Time-travel debugging lets you jump your live running app to any past state — without refreshing the page, without changing any code.

How to Use It

Hover over any action in the log. Two buttons appear on the right side of that row:



Click "Jump" on the `@@INIT` entry at the very top.

Your app's UI immediately reflects the initial state — the form is empty, the summary page shows nothing, the details page shows dashes. **Your components are re-rendering with the past state, live, right now.** You did not refresh the page. You did not change any code. You just moved back in time.

Click "Jump" on the `[Profile Form] Save Profile` entry — the app jumps forward to the state after the save. The summary page shows the name again instantly.

Why This Changes How You Debug

Imagine a user reports: "*I filled in the form, navigated to the details page, clicked something, and my name disappeared.*"

Without DevTools	With DevTools
Add `console.log` everywhere	Open the action log — it is already there
Try to reproduce the exact steps	Click Diff on each action — find where `name` became `""`
Guess which component caused it	See the action type — e.g. `[Profile Form] Clear Profile`
Add more logs, rebuild, retry	Jump to that moment in time. Bug confirmed and located in under 30 seconds.

---

8 The Skip Button

The **Skip** button next to each action temporarily removes that action from the state calculation — as if it never happened.

Example scenario — you have dispatched Save twice:

```
@@INIT
[Profile Form] Save Profile   ← first save (name: "Reza")
[Profile Form] Save Profile   ← second save (name: "Ahmed")
```

Click **Skip** on the first Save. NgRx recalculates the state as if only the second Save ever ran. Your app now shows "Ahmed" — the result of only that one action. Un-skip it and the first Save comes back into the calculation.

> **What is Skip useful for?**
> It lets you ask: "*What would my app look like if this one specific action had never fired?*" This is invaluable for testing edge cases and understanding the cumulative effect of a sequence of actions — without writing a single test or restarting the app.

---

9 The State Tree Explorer

At the top of the right panel there is a **State** tab that shows the *current live state* of the entire store — not tied to any specific action, just always up to date.

It renders as a collapsible tree:

```
▼ userProfile
   name: "Reza"
   email: "reza@example.com"
   age: 25
```

As you interact with your app, this tree updates in real time. When you add more feature slices later (products, cart, auth…) they appear as separate collapsible branches at the same level as `userProfile`. This gives you a complete live map of your entire application state.

---

10 Production Safety

You already handled the basics with `logOnly: !isDevMode()` in `app.config.ts`. Here is what that means in full:

Command	`isDevMode()`	`logOnly`	DevTools behaviour
`ng serve`	`true`	`false`	Fully active — time-travel, skip, jump all work
`ng build`	`false`	`true`	Read-only — extension connects but cannot modify state

For maximum security in production — where you do not want to expose any state to the browser extension at all — skip registering DevTools entirely when not in dev mode:

`src/app/app.config.ts — maximum safety version`

```
import { ApplicationConfig, isDevMode } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideStore } from '@ngrx/store';
import { provideStoreDevtools } from '@ngrx/store-devtools';

import { routes } from './app.routes';
import { userProfileReducer } from './state/user-profile.reducer';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideStore({ userProfile: userProfileReducer }),

    // Spread an array conditionally:
    // In dev  → array has one item  → DevTools is registered
    // In prod → array is empty      → DevTools is not registered at all
    ...(isDevMode()
      ? [provideStoreDevtools({ maxAge: 25, name: 'NgRx User Profile App' })]
```

```
      : [])
  ],
};
```

> **Never ship DevTools fully active to production.** The DevTools extension exposes your entire store state in the browser. A user with the extension installed could inspect — and potentially manipulate — all your application state. Always use `logOnly: !isDevMode()` as a minimum, or the conditional spread pattern above for maximum safety.

Quick Reference — DevTools Cheat Sheet

What you want to do	How
See every action that fired in order	Left panel — the action log
See the data payload inside an action	Click the action → **Action** tab
See the full store state after an action	Click the action → **State** tab
See only what changed	Click the action → **Diff** tab
Jump to a past state (time-travel)	Hover any action → click **Jump**
Remove one action from the calculation	Hover any action → click **Skip**
See the live current state tree	**State** tab at the top right of the panel
Clear the entire action log	**Reset** button at the top of the left panel
Open the Redux tab quickly	`F12` → >> → Redux

Setup Summary — Everything in One Place

```
# Step 1: Install the browser extension
# Chrome Web Store → search "Redux DevTools" → Add to Chrome

# Step 2: Install the npm package
```

```
npm install @ngrx/store-devtools@17

# Step 3: Add to app.config.ts
# import { provideStoreDevtools } from '@ngrx/store-devtools';
# import { isDevMode } from '@angular/core';
#
# providers: [
#   ...
#   provideStoreDevtools({ maxAge: 25, logOnly: !isDevMode() }),
# ]

# Step 4: Run the app
ng serve

# Step 5: Open the panel
# F12 → >> → Redux
```

You are done. Every action your app dispatches is now logged, inspectable, diffable, and rewindable — for the entire browser session.

Once you debug with DevTools for a week, going back to `console.log` feels like navigating with a paper map.